EXP 26

Q1. Create a program using Runnable to run two tasks simultaneously.

Java Code:

```
class TaskOne implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("TaskOne prints: " + i);
        }
    }
}

class TaskTwo implements Runnable {
    public void run() {
        for (int i = 10; i <= 15; i++) {
            System.out.println("TaskTwo prints: " + i);
        }
    }
}

public class TwoTasksDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread(new TaskOne(), "Thread-1");
        Thread t2 = new Thread(new TaskTwo(), "Thread-2");

        t1.start();
        t2.start();
    }
}
```

Q2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

Java Code:

```
class BankAccount {
   private int balance = 1000;

   public synchronized void deposit(int amount) {
      balance += amount;
      System.out.println(Thread.currentThread().getName() +
            " deposited: " + amount + " | Balance: " + balance);
   }

   public synchronized void withdraw(int amount) {
      if (balance >= amount) {
         balance -= amount;
         System.out.println(Thread.currentThread().getName() +
               " withdrew: " + amount + " | Balance: " + balance);
      } else {
         System.out.println(Thread.currentThread().getName() +
               " tried to withdraw " + amount + " | Insufficient balance!");
      }
   }
}

class DepositTask implements Runnable {
   private BankAccount account;
   public DepositTask(BankAccount account) { this.account = account; }
   public void run() {
```

```
        for (int i = 1; i <= 3; i++) {
            account.deposit(200);
        }
    }
}

class WithdrawTask implements Runnable {
    private BankAccount account;
    public WithdrawTask(BankAccount account) { this.account =
account; }
    public void run() {
        for (int i = 1; i <= 3; i++) {
            account.withdraw(150);
        }
    }
}

public class BankTransactionDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        Thread depositThread = new Thread(new DepositTask(account),
"DepositThread");
        Thread withdrawThread = new Thread(new
WithdrawTask(account), "WithdrawThread");

        depositThread.start();
        withdrawThread.start();
    }
}
```

Q1 Output

```
TaskOne prints: 1
TaskTwo prints: 10
TaskOne prints: 2
TaskTwo prints: 11
TaskOne prints: 3
TaskTwo prints: 12
TaskOne prints: 4
TaskTwo prints: 13
TaskOne prints: 5
TaskTwo prints: 14
TaskTwo prints: 15
```

Q2 Output

```
DepositThread deposited: 200 | Balance: 1200
WithdrawThread withdrew: 150 | Balance: 1050
DepositThread deposited: 200 | Balance: 1250
WithdrawThread withdrew: 150 | Balance: 1100
DepositThread deposited: 200 | Balance: 1300
WithdrawThread withdrew: 150 | Balance: 1150
```